

Manipulação Avançada de Arrays e Operações Numéricas com NumPy

Objetivo da aula

A proposta desta etapa é expandir o uso do NumPy para além da criação simples de arrays, mostrando como essa biblioteca passa a atuar como uma ferramenta central para processamento numérico em Python. O foco deixa de ser apenas “criar estruturas com números” e passa a ser “manipular dados numéricos com eficiência, clareza e aplicação prática”.

Num contexto profissional, arrays não existem apenas para guardar valores. Eles são usados para:

- processar medições de sensores;
- calcular médias, máximos e mínimos de grandes conjuntos de dados;
- organizar leituras numéricas em forma de vetores e matrizes;
- transformar estruturas de dados para análise;
- executar operações matemáticas de forma massiva;
- servir de base para bibliotecas mais avançadas como Pandas, Scikit-Learn e TensorFlow.

O NumPy é uma biblioteca construída exatamente para esse cenário. Enquanto listas do Python são genéricas e flexíveis, arrays NumPy são especializados em números e foram criados para lidar com cálculo, estrutura e desempenho.

1. Consolidação da instalação do NumPy no ambiente virtual

Antes de qualquer uso prático da biblioteca, é necessário que o projeto esteja trabalhando dentro do ambiente virtual. Esse detalhe não é apenas operacional; ele faz parte da organização profissional do projeto.

Quando uma biblioteca externa é instalada sem ambiente virtual, ela passa a ficar disponível de forma global na máquina. Isso pode gerar conflitos entre projetos, especialmente quando um sistema depende de uma versão diferente da mesma biblioteca.

Por isso, a ordem correta de trabalho é:

1. abrir a pasta do projeto no VS Code;
2. ativar o ambiente virtual;
3. instalar a biblioteca dentro desse ambiente;
4. usar a biblioteca no projeto.

Ativação do ambiente virtual

No Windows, com o terminal aberto na pasta do projeto:

```
Python -m venv venv
```

```
venv\Scripts\activate
```

Quando a ativação ocorre corretamente, o terminal costuma exibir algo semelhante a:

(venv) C:\caminho\do\projeto>

Essa indicação é importante porque mostra que o terminal não está mais usando o Python global da máquina, mas sim o Python isolado daquele projeto.

Instalação do NumPy

Com o ambiente ativo, a instalação ocorre com:

```
pip install numpy
```

Explicação detalhada do comando

- pip é o gerenciador de pacotes do Python;
- install indica que um pacote será instalado;
- numpy é o nome da biblioteca que será baixada e adicionada ao ambiente virtual.

Ao executar esse comando, o Python instala a biblioteca dentro do venv, e não no sistema todo. Isso significa que o NumPy passa a pertencer àquele projeto.

Conferência da instalação

Para verificar se a biblioteca realmente foi instalada:

```
pip list
```

ou

```
pip show numpy
```

O segundo comando é particularmente útil porque mostra informações detalhadas como:

- nome da biblioteca;
- versão instalada;
- local onde foi instalada.

Isso reforça a compreensão de que a biblioteca foi integrada ao ambiente isolado do projeto.

2. O que significa computação numérica em Python

Computação numérica é o uso da programação para manipular, transformar, organizar e calcular grandes volumes de dados numéricos.

Essa área aparece em praticamente todos os campos técnicos que lidam com dados quantitativos. Alguns exemplos:

- em engenharia, para cálculos estruturais, medições e simulações;
- em estatística, para resumir, comparar e interpretar dados;
- em ciência de dados, para preparar bases numéricas antes da análise;

- em inteligência artificial, para trabalhar com vetores, matrizes e tensores;
- em sistemas de automação e IoT, para tratar leituras de sensores.

Em Python puro, é possível trabalhar com números usando variáveis, listas, laços de repetição e funções. O problema aparece quando a quantidade de dados cresce ou quando as operações precisam ser repetidas em massa.

Exemplo com lista Python:

```
temperaturas = [22, 24, 23, 25, 21, 26]

resultado = []
for temperatura in temperaturas:
    resultado.append(temperatura + 1)

print(resultado)
```

O código funciona, mas depende de laço, criação manual de nova lista e processamento explícito.

Agora observe a mesma ideia com NumPy:

```
import numpy as np

temperaturas = np.array([22, 24, 23, 25, 21, 26])

resultado = temperaturas + 1

print(resultado)
```

A lógica ficou mais curta, mais clara e mais próxima de um raciocínio matemático.

O papel do NumPy começa exatamente aqui: tornar o tratamento de dados numéricos mais eficiente e mais expressivo.

3. Importação da biblioteca e padrão da comunidade

Depois de instalada, a biblioteca precisa ser importada no código.

O padrão universalmente adotado é:

```
import numpy as np
```

O que esse comando faz

- import é a palavra reservada usada para trazer uma biblioteca ou módulo para dentro do arquivo atual;
- numpy é o nome oficial da biblioteca;
- as np define um apelido, também chamado de alias.

Esse apelido não é aleatório. np virou convenção da comunidade Python. Isso significa que praticamente toda documentação, curso, tutorial, projeto real ou código técnico usa np ao se referir ao NumPy.

Exemplo:

```
import numpy as np

dados = np.array([1, 2, 3])
print(dados)
```

Sem alias, seria necessário escrever:

```
import numpy

dados = numpy.array([1, 2, 3])
print(dados)
```

As duas formas funcionam, mas a primeira é mais padronizada e mais prática.

4. Diferença real entre lista Python e array NumPy

Esse ponto precisa ser compreendido com cuidado, porque não se trata apenas de “duas estruturas parecidas”.

Lista Python

Uma lista Python é uma estrutura genérica. Ela foi projetada para guardar vários tipos de elementos. Exemplo:

```
dados = [10, "Maria", True, 8.5]

print(dados)
```

Essa flexibilidade é útil, mas tem custo. Como a estrutura é genérica, ela não é especializada para processamento matemático intensivo.

Array NumPy

Um array NumPy é uma estrutura voltada para dados homogêneos, normalmente numéricos. Exemplo:

```
import numpy as np

dados = np.array([10, 20, 30, 40])

print(dados)
```

O array NumPy foi otimizado para:

- armazenar números de forma mais eficiente;
- executar operações matemáticas em lote;
- trabalhar com várias dimensões;
- servir de base para computação numérica.

Comparação prática

Lista Python

```
numeros = [1, 2, 3, 4, 5]
print(numeros)
```

Array NumPy

```
import numpy as np

numeros_np = np.array([1, 2, 3, 4, 5])
print(numeros_np)
```

Ambos exibem uma sequência de valores. A diferença aparece quando se tenta fazer cálculo.

Multiplificação em lista Python

```
numeros = [1, 2, 3, 4, 5]
resultado = []

for numero in numeros:
    resultado.append(numero * 2)

print(resultado)
```

Multiplificação em array NumPy

```
import numpy as np

numeros_np = np.array([1, 2, 3, 4, 5])
resultado = numeros_np * 2

print(resultado)
```

No segundo caso, o cálculo foi aplicado automaticamente a todos os elementos.

Essa diferença não é apenas sintática. Ela representa uma mudança de paradigma:

- com listas, pensa-se em percorrer elementos;
- com arrays NumPy, pensa-se em operar sobre a estrutura inteira.

5. Operações vetorizadas — o conceito central do NumPy

O conceito mais importante desta aula é o de **operação vetorizada**.

Vetorização significa aplicar uma operação diretamente sobre todos os elementos de um array sem precisar escrever laços explícitos como for.

Isso faz o código ficar:

- mais curto;
- mais legível;
- mais próximo da matemática;
- mais eficiente.

Exemplo básico

```
import numpy as np

numeros = np.array([1, 2, 3, 4, 5])

resultado = numeros + 10

print(resultado)
```

Saída esperada:

```
[11 12 13 14 15]
```

O que aconteceu

O NumPy não somou 10 apenas ao array inteiro como um objeto único. Ele percorreu internamente todos os elementos e aplicou a soma um a um.

Ou seja:

- $1 + 10 = 11$
- $2 + 10 = 12$
- $3 + 10 = 13$
- $4 + 10 = 14$
- $5 + 10 = 15$

Tudo isso sem for.

Exemplo intermediário — ajuste de preços

Imagine um sistema simples de análise de produtos em que todos os preços sofreram aumento de 5 reais.

```
import numpy as np

precos = np.array([50.0, 80.0, 120.0, 200.0])
```

```
novos_precos = precos + 5
```

```
print("Preços antigos:", precos)  
print("Preços atualizados:", novos_precos)
```

Esse tipo de lógica é muito comum em tratamento de dados em lote.

Exemplo avançado — reajuste percentual

```
import numpy as np
```

```
salarios = np.array([1800.0, 2500.0, 3200.0, 4100.0])
```

```
salarios_reajustados = salarios * 1.08
```

```
print("Salários originais:", salarios)  
print("Salários com reajuste de 8%:", salarios_reajustados)
```

Aqui, cada salário recebeu 8% de reajuste de uma só vez.

6. Operações matemáticas completas com arrays

Arrays NumPy permitem aplicar operações aritméticas básicas diretamente.

Soma

```
import numpy as np
```

```
dados = np.array([10, 20, 30])
```

```
print(dados + 5)
```

Saída:

```
[15 25 35]
```

Subtração

```
import numpy as np
```

```
dados = np.array([10, 20, 30])
```

```
print(dados - 5)
```

Saída:

```
[ 5 15 25]
```

Multiplicação

```
import numpy as np

dados = np.array([10, 20, 30])

print(dados * 2)

Saída:

[20 40 60]
```

Divisão

```
import numpy as np

dados = np.array([10, 20, 30])

print(dados / 2)

Saída:

[ 5. 10. 15.]
```

Potenciação

```
import numpy as np

dados = np.array([2, 3, 4])

print(dados ** 2)

Saída:

[ 4  9 16]
```

Resto da divisão

```
import numpy as np

dados = np.array([10, 11, 12, 13])

print(dados % 2)

Saída:

[0 1 0 1]
```

Esse tipo de operação é útil, por exemplo, para separar números pares e ímpares, analisar periodicidade ou verificar padrões de ciclos.

7. Operações entre arrays

Quando dois arrays possuem o mesmo formato, o NumPy permite que eles interajam elemento por elemento.

Soma entre arrays

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
print(a + b)
```

Saída:

```
[5 7 9]
```

Interpretação:

- posição 0: 1 + 4
- posição 1: 2 + 5
- posição 2: 3 + 6

Multiplificação entre arrays

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
print(a * b)
```

Saída:

```
[ 4 10 18]
```

Exemplo aplicado — vendas de filiais

Imagine duas filiais vendendo o mesmo produto em quantidades diárias diferentes.

```
import numpy as np
```

```
filial_a = np.array([10, 15, 12, 20, 18])
```

```
filial_b = np.array([8, 14, 11, 19, 17])
```

```
vendas_totais = filial_a + filial_b
```

```
print("Filial A:", filial_a)
```

```
print("Filial B:", filial_b)
```

```
print("Total diário das duas filiais:", vendas_totais)
```

Agora a soma possui contexto empresarial claro.

Exemplo aplicado — consumo e preço unitário

```
import numpy as np
```

```
consumo_kwh = np.array([120, 150, 180, 200])
```

```
preco_unitario = np.array([0.80, 0.80, 0.80, 0.80])
```

```
custo = consumo_kwh * preco_unitario
```

```
print("Consumo em kWh:", consumo_kwh)  
print("Preço unitário:", preco_unitario)  
print("Custo por registro:", custo)
```

Aqui, a multiplicação elemento a elemento representa cálculo de valor por consumo.

8. Geração automática de dados com `arange()`

A função `arange()` gera uma sequência numérica automaticamente.

Forma simples

```
import numpy as np  
  
numeros = np.arange(10)  
  
print(numeros)
```

Saída:

```
[0 1 2 3 4 5 6 7 8 9]
```

Explicação

Quando apenas um argumento é fornecido, o NumPy entende que:

- começa em 0;
- vai até o valor informado;
- não inclui o limite final.

Ou seja:

```
np.arange(10)
```

significa, na prática:

0 até 9

Forma com início e fim

```
import numpy as np  
  
numeros = np.arange(5, 15)  
  
print(numeros)
```

Saída:

```
[ 5  6  7  8  9 10 11 12 13 14]
```

Forma com passo

```
import numpy as np

numeros = np.arange(0, 21, 5)

print(numeros)
```

Saída:

```
[ 0  5 10 15 20]
```

Explicação dos parâmetros

`arange(inicio, fim, passo)`

- `inicio`: valor inicial da sequência;
- `fim`: limite final, não incluído;
- `passo`: intervalo entre um valor e outro.

Exemplo aplicado — minutos de coleta

```
import numpy as np

minutos = np.arange(0, 61, 10)

print("Instantes de coleta em minutos:", minutos)
```

Esse tipo de sequência pode representar coletas automáticas de sensor ao longo de uma hora.

9. Geração automática de dados com `linspace()`

A função `linspace()` cria números igualmente espaçados entre dois valores.

Exemplo básico

```
import numpy as np

valores = np.linspace(0, 10, 5)

print(valores)
```

Saída:

```
[ 0.  2.5  5.  7.5 10.]
```

Explicação do comando

`linspace(inicio, fim, quantidade)`

- `inicio`: valor inicial;
- `fim`: valor final;
- `quantidade`: número de pontos que devem ser criados.

Diferentemente de `arange()`, que trabalha com passo, `linspace()` trabalha com quantidade total de pontos.

Exemplo aplicado — marcação de pontos em um intervalo

```
import numpy as np

tensoes = np.linspace(0, 5, 6)

print("Pontos de tensão analisados:", tensoes)
```

Saída:

```
[0. 1. 2. 3. 4. 5.]
```

Exemplo mais elaborado — amostragem de intervalo

```
import numpy as np

amostras = np.linspace(100, 200, 11)

print("Amostras igualmente espaçadas:", amostras)
```

Esse tipo de estrutura é comum em gráficos, simulações e modelagens matemáticas.

10. Criação de arrays especiais

NumPy possui funções prontas para gerar arrays padronizados. Essas estruturas são muito usadas em simulações, inicialização de dados, testes e organização matemática.

zeros()

Cria um array preenchido com zeros.

```
import numpy as np

zeros = np.zeros(5)

print(zeros)
```

Saída:

```
[0. 0. 0. 0. 0.]
```

O que isso representa

Um conjunto de cinco posições já inicializadas com zero.

Exemplo aplicado — contador inicial

```
import numpy as np

registros_por_maquina = np.zeros(4)

print("Registros iniciais:", registros_por_maquina)
```

Isso pode representar quatro máquinas que ainda não registraram nenhuma produção.

ones()

Cria um array preenchido com uns.

```
import numpy as np
```

```
uns = np.ones(5)
```

```
print(uns)
```

Saída:

```
[1. 1. 1. 1. 1.]
```

Exemplo aplicado — pesos iniciais iguais

```
import numpy as np
```

```
pesos_iniciais = np.ones(3)
```

```
print("Pesos iniciais:", pesos_iniciais)
```

Matrizes com zeros

```
import numpy as np
```

```
matriz = np.zeros((3, 3))
```

```
print(matriz)
```

Saída:

```
[[0. 0. 0.]
```

```
 [0. 0. 0.]
```

```
 [0. 0. 0.]]
```

Explicação do (3, 3)

O argumento foi passado como uma tupla:

- primeiro valor: quantidade de linhas;
- segundo valor: quantidade de colunas.

Portanto:

(3, 3)

significa:

- 3 linhas
- 3 colunas

Matrizes com uns

```
import numpy as np

matriz = np.ones((2, 4))

print(matriz)
```

Saída:

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

11. Estatística com NumPy

Um dos grandes benefícios do NumPy é já trazer operações estatísticas prontas, o que evita cálculos manuais desnecessários.

Considere o seguinte conjunto de notas:

```
import numpy as np

notas = np.array([7.5, 8.0, 6.5, 9.0])

print(np.mean(notas))
print(np.sum(notas))
print(np.max(notas))
print(np.min(notas))
```

Média com mean()

```
print(np.mean(notas))
```

Saída:

7.75

- **O que faz:** Calcula a média aritmética dos valores.

Soma com sum()

```
print(np.sum(notas))
```

Saída:

31.0

- **O que faz:** Soma todos os elementos do array.

Máximo com max()

```
print(np.max(notas))
```

Saída:

9.0

- **O que faz:** Retorna o maior valor presente no array.

Mínimo com min()

```
print(np.min(notas))
```

Saída:

6.5

- **O que faz:** Retorna o menor valor do array.

Exemplo aplicado — monitoramento de temperatura

```
import numpy as np
```

```
temperaturas = np.array([22, 24, 23, 25, 21, 26, 24])
```

```
media = np.mean(temperaturas)
```

```
maxima = np.max(temperaturas)
```

```
minima = np.min(temperaturas)
```

```
soma = np.sum(temperaturas)
```

```
print("Leituras:", temperaturas)
```

```
print("Média:", media)
```

```
print("Máxima:", maxima)
```

```
print("Mínima:", minima)
```

```
print("Soma total das leituras:", soma)
```

Esse tipo de análise é um exemplo claro de uso prático da biblioteca.

12. Manipulação de dimensões com reshape()

Agora entra um dos pontos mais importantes da aula, que realmente precisava ser tratado com profundidade: **transformação estrutural de arrays**.

- `reshape()` serve para reorganizar a forma de um array sem alterar seus dados.

Exemplo inicial

```
import numpy as np
```

```
dados = np.arange(6)
```

```
print(dados)
```

Saída:

[0 1 2 3 4 5]

Esse array possui 6 elementos em uma dimensão.

Agora, reorganizando:

```
matriz = dados.reshape(2, 3)
```

```
print(matriz)
```

Saída:

```
[[0 1 2]  
 [3 4 5]]
```

O que aconteceu

Para o NumPy, redimensionar um array não significa criar ou deletar dados, mas sim reorganizar a visualização dos elementos existentes na memória. Por isso, a regra de ouro é: A quantidade total de elementos original deve ser exatamente igual ao produto das novas dimensões.

⇒ Por que isso é obrigatório?

O NumPy aloca um bloco contínuo de memória para os seus dados. Quando você muda o formato (o *shape*), você está apenas mudando a "grade" (linhas e colunas) que sobrepõe esses dados. Se a conta não fechar, sobrariam espaços vazios ou faltariam números para preencher a grade, o que o Python não permite.

⇒ Exemplos Práticos de Compatibilidade (Total = 6)

Se você tem um array com 6 elementos, a multiplicação das novas dimensões precisa resultar em 6:

- **2x3 (2 linhas e 3 colunas):** Perfeito: os 6 elementos se encaixam em duas linhas de três.
- **3x2 (3 linhas e 2 colunas):** Perfeito: os 6 elementos se encaixam em três linhas de dois.
- **1x6 (1 linha e 6 colunas):** Transforma o array em uma matriz linha.
- **6x1 (6 linhas e 1 coluna):** Transforma o array em uma matriz coluna.

⇒ Por que o Exemplo Incorreto falha?

- Tentar virar 4x2:
 - Cálculo: $4 \times 2 = 8$
 - **Problema:** Você tem apenas **6** peças de informação, mas está tentando montar um quebra-cabeça que exige **8** peças. Como o NumPy não "inventa" dados (como zeros ou valores nulos) por conta própria no reshape, ele retornará um erro de valor (ValueError).

Exemplo com 12 elementos

```
import numpy as np
```

```
dados = np.arange(12)
```

```
print("Vetor original:")  
print(dados)
```

```
matriz = dados.reshape(3, 4)
```

```
print("Matriz 3x4:")
print(matriz)
```

Saída esperada:

Vetor original:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Matriz 3x4:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Exemplo aplicado — relatório mensal em semanas

Imagine 12 valores representando desempenho semanal distribuído em 3 períodos com 4 registros cada.

```
import numpy as np
```

```
producao = np.array([100, 120, 110, 130, 140, 150, 145, 155, 160, 162, 158, 170])
```

```
matriz_producao = producao.reshape(3, 4)
```

```
print("Produção organizada em matriz:")
print(matriz_producao)
```

Aqui, o `reshape()` ajuda a reorganizar dados para leitura analítica.

Vou continuar aprofundando indexação, slicing, seleção por condição, operações mais realistas e exemplos intermediários e avançados na sequência natural abaixo.

13. Indexação em arrays NumPy

Indexação é o acesso direto a elementos específicos dentro do array.

Esse conceito já existe em listas Python, mas em NumPy ele se torna ainda mais importante porque será usado tanto em vetores quanto em matrizes.

Acesso em vetor unidimensional

```
import numpy as np
```

```
dados = np.array([10, 20, 30, 40, 50])
```

```
print(dados[0])
print(dados[2])
print(dados[4])
```

Saída:

10
30
50

Explicação

O índice sempre começa em zero:

- dados[0] → primeiro elemento
- dados[1] → segundo elemento
- dados[2] → terceiro elemento

Índices negativos

NumPy também aceita índices negativos, acessando o array a partir do fim.

```
import numpy as np
```

```
dados = np.array([10, 20, 30, 40, 50])
```

```
print(dados[-1])  
print(dados[-2])
```

Saída:

50
40

Explicação

- -1 → último elemento
- -2 → penúltimo elemento

Esse recurso é útil quando o interesse está nos dados mais recentes ou na última posição de uma estrutura.

Exemplo aplicado — leitura do último registro

```
import numpy as np
```

```
leituras = np.array([18.2, 18.5, 18.7, 19.0, 19.3])
```

```
ultima_leitura = leituras[-1]
```

```
print("Última leitura registrada:", ultima_leitura)
```

14. Slicing em arrays NumPy

Slicing é o acesso a faixas de elementos.

Em vez de pegar um único valor, pega-se um conjunto contínuo de posições.

Exemplo básico

```
import numpy as np

dados = np.array([10, 20, 30, 40, 50])

print(dados[1:4])
```

Saída:

```
[20 30 40]
```

Explicação detalhada

A estrutura do slicing é:

```
array[inicio:fim]
```

- início é incluído
- fim não é incluído

Então:

```
dados[1:4]
```

significa:

- começa no índice 1
- vai até antes do índice 4

Logo, pega:

- índice 1 → 20
- índice 2 → 30
- índice 3 → 40

Slicing do começo até um ponto

```
import numpy as np

dados = np.array([10, 20, 30, 40, 50])

print(dados[:3])
```

Saída:

```
[10 20 30]
```

Isso significa: do início até antes do índice 3.

Slicing de um ponto até o final

```
import numpy as np

dados = np.array([10, 20, 30, 40, 50])

print(dados[2:])

Saída:

[30 40 50]
```

Slicing com passo

```
import numpy as np

dados = np.array([10, 20, 30, 40, 50, 60])

print(dados[::2])

Saída:

[10 30 50]
```

Explicação

A sintaxe completa do slicing é:

```
array[inicio:fim:passo]
```

No exemplo:

- início omitido → começa do início
- fim omitido → vai até o final
- passo 2 → pega de dois em dois

Exemplo aplicado — separação de registros alternados

```
import numpy as np

medicoes = np.array([101, 103, 102, 104, 105, 107, 106, 108])

amostras_alternadas = medicoes[::2]

print("Medições originais:", medicoes)
print("Medições alternadas:", amostras_alternadas)
```

Esse tipo de seleção pode ser útil em amostragem, testes ou filtrações periódicas.

15. Indexação e slicing em matrizes

Quando o array possui duas dimensões, o acesso passa a ocorrer por linha e coluna.

Exemplo de matriz

```
import numpy as np
```

```
matriz = np.array([  
    [10, 20, 30],  
    [40, 50, 60],  
    [70, 80, 90]  
])
```

```
print(matriz)
```

Acessando um elemento específico

```
print(matriz[0, 0])  
print(matriz[1, 2])  
print(matriz[2, 1])
```

Saída:

```
10  
60  
80
```

Explicação

A estrutura é:

```
matriz[linha, coluna]
```

Então:

- `matriz[0, 0]` → linha 0, coluna 0
- `matriz[1, 2]` → linha 1, coluna 2
- `matriz[2, 1]` → linha 2, coluna 1

Acessando uma linha inteira

```
print(matriz[1])
```

Saída:

```
[40 50 60]
```

Acessando uma coluna inteira

```
print(matriz[:, 1])
```

Saída:

```
[20 50 80]
```

Explicação do :

O símbolo “:” significa “todos” naquele eixo.

Logo:

matriz[:, 1]

significa:

- todas as linhas
- coluna 1

Exemplo aplicado — vendas mensais por setor

```
import numpy as np
```

```
vendas = np.array([
    [1200, 1500, 1700],
    [1100, 1400, 1600],
    [1300, 1550, 1800]
])
```

```
print("Tabela de vendas:")
print(vendas)
```

```
print("Primeira linha de vendas:", vendas[0])
print("Segunda coluna de vendas:", vendas[:, 1])
print("Elemento da linha 2, coluna 3:", vendas[1, 2])
```

Esse tipo de estrutura já se aproxima muito da forma como dados empresariais podem ser organizados numericamente.

16. Seleção condicional com NumPy

Uma das aplicações mais úteis de NumPy é a seleção de valores com base em condições.

Exemplo básico

```
import numpy as np
```

```
temperaturas = np.array([22, 24, 23, 25, 21, 26])
```

```
maskara = temperaturas > 23
```

```
print(maskara)
```

Saída:

```
[False True False True False True]
```

O que aconteceu

NumPy comparou cada valor do array com 23 e gerou um novo array de booleanos.

- valores maiores que 23 → True
- valores menores ou iguais a 23 → False

Usando a condição para filtrar valores

```
import numpy as np

temperaturas = np.array([22, 24, 23, 25, 21, 26])

altas = temperaturas[temperaturas > 23]

print(altas)

Saída:

[24 25 26]
```

Exemplo aplicado — vendas acima da meta

```
import numpy as np

vendas = np.array([800, 1200, 950, 1500, 1100, 700])

vendas_acima_da_meta = vendas[vendas > 1000]

print("Vendas registradas:", vendas)
print("Vendas acima da meta:", vendas_acima_da_meta)
```

Esse tipo de filtragem é muito útil em análise de dados, monitoramento e estatística.

Exemplo com múltiplas condições

```
import numpy as np

idades = np.array([15, 18, 22, 30, 17, 19, 45])

faixa = idades[(idades >= 18) & (idades <= 30)]

print("Idades na faixa selecionada:", faixa)
```

Explicação importante

Quando se usa NumPy com múltiplas condições, utiliza-se:

- & para “e”
- | para “ou”

E cada comparação deve estar entre parênteses.

Isso é essencial para evitar erro de precedência.

17. Funções estatísticas mais ricas

Além de média, soma, máximo e mínimo, NumPy oferece outras operações muito úteis.

Média

```
import numpy as np

dados = np.array([10, 20, 30, 40, 50])
```

```
print(np.mean(dados))
```

Mediana

```
import numpy as np
```

```
dados = np.array([10, 20, 30, 40, 50])
```

```
print(np.median(dados))
```

A mediana representa o valor central do conjunto ordenado.

Desvio padrão

```
import numpy as np
```

```
dados = np.array([10, 20, 30, 40, 50])
```

```
print(np.std(dados))
```

O que é desvio padrão

É uma medida estatística que mostra o quanto os dados variam em relação à média.

Quanto maior o desvio padrão, maior a dispersão dos valores.

Exemplo aplicado — análise de desempenho

```
import numpy as np
```

```
tempos = np.array([12.5, 13.0, 12.8, 13.1, 12.9, 13.5, 12.7])
```

```
print("Tempos:", tempos)
```

```
print("Média:", np.mean(tempos))
```

```
print("Mediana:", np.median(tempos))
```

```
print("Desvio padrão:", np.std(tempos))
```

```
print("Maior tempo:", np.max(tempos))
```

```
print("Menor tempo:", np.min(tempos))
```

Esse tipo de resumo estatístico é típico em análise de produção, sensores, desempenho de sistemas e experimentos.

18. Exemplo prático completo — monitoramento de temperaturas

Agora é importante sair de exemplos isolados e montar um bloco mais próximo de uso real.

Suponha que uma empresa esteja monitorando a temperatura de um ambiente técnico ao longo de vários horários do dia.

```
import numpy as np
```

```
temperaturas = np.array([22.1, 23.4, 24.0, 25.2, 24.8, 23.9, 22.7, 21.9])
```

```
media = np.mean(temperaturas)
maxima = np.max(temperaturas)
minima = np.min(temperaturas)
amplitude = maxima - minima
acima_de_24 = temperaturas[temperaturas > 24]

print("Temperaturas registradas:", temperaturas)
print("Temperatura média:", media)
print("Temperatura máxima:", maxima)
print("Temperatura mínima:", minima)
print("Amplitude térmica:", amplitude)
print("Temperaturas acima de 24 graus:", acima_de_24)
```

Leitura técnica do exemplo

- `np.array(...)` cria a estrutura numérica;
- `np.mean(...)` calcula a média;
- `np.max(...)` encontra o maior valor;
- `np.min(...)` encontra o menor valor;
- `maxima - minima` calcula a amplitude;
- `temperaturas[temperaturas > 24]` seleciona apenas os valores acima de 24.

Esse exemplo mostra que NumPy não serve apenas para “guardar números”, mas para fazer análise rápida sobre eles.

19. Exemplo prático completo — análise de produção em uma semana

```
import numpy as np

producao = np.array([120, 135, 128, 142, 150, 138, 145])

media = np.mean(producao)
maior = np.max(producao)
menor = np.min(producao)
dias_acima_da_media = producao[producao > media]
producao_reajustada = producao + 10

print("Produção diária:", producao)
print("Média semanal:", media)
print("Maior produção:", maior)
print("Menor produção:", menor)
print("Dias acima da média:", dias_acima_da_media)
print("Produção reajustada (+10):", producao_reajustada)
```

O que esse exemplo consolida

Nesse único bloco aparecem vários conteúdos da aula:

- array numérico;
- média;
- máximo;
- mínimo;
- seleção condicional;
- operação vetorizada.

Isso demonstra como os conceitos da aula se conectam naturalmente.

20. Exemplo prático com matriz — desempenho de três setores

```
import numpy as np

desempenho = np.array([
    [100, 120, 130, 140],
    [90, 110, 125, 135],
    [95, 115, 128, 138]
])

print("Matriz de desempenho:")
print(desempenho)

print("Formato da matriz:", desempenho.shape)
print("Número de dimensões:", desempenho.ndim)
print("Quantidade de elementos:", desempenho.size)

print("Primeira linha:", desempenho[0])
print("Segunda coluna:", desempenho[:, 1])

media_geral = np.mean(desempenho)
maior_valor = np.max(desempenho)
menor_valor = np.min(desempenho)

print("Média geral:", media_geral)
print("Maior valor:", maior_valor)
print("Menor valor:", menor_valor)
```

Esse tipo de organização matricial é muito relevante porque aproxima o uso de NumPy de tabelas numéricas reais, mesmo antes de entrar formalmente em bibliotecas como Pandas.

21. Exemplo avançado coerente — reorganização e análise conjunta

```
import numpy as np

leituras = np.arange(1, 13)

print("Leituras lineares:")
print(leituras)

matriz = leituras.reshape(3, 4)
```

```
print("Leituras organizadas em matriz 3x4:")
print(matriz)

print("Segunda linha da matriz:", matriz[1])
print("Terceira coluna da matriz:", matriz[:, 2])

media_total = np.mean(matriz)
soma_total = np.sum(matriz)

print("Média total:", media_total)
print("Soma total:", soma_total)
```

O que esse código demonstra

- geração automática com `arange`;
- reorganização com `reshape`;
- acesso por linha;
- acesso por coluna;
- estatística em estrutura bidimensional.

Ou seja, vários recursos da aula foram integrados em um único contexto técnico coerente.

22. Erros conceituais comuns ao trabalhar com NumPy

É importante prevenir alguns erros de raciocínio.

Erro 1 — achar que array NumPy é apenas uma lista “mais bonita”

Não é. Array NumPy é uma estrutura especializada para dados numéricos, com comportamento próprio e otimização interna.

Erro 2 — esquecer compatibilidade no `reshape()`

Exemplo incorreto:

```
import numpy as np
```

```
dados = np.arange(6)
```

```
matriz = dados.reshape(4, 2)
```

Esse código gera erro porque:

- o vetor possui 6 elementos;
- a forma 4x2 exigiria 8 elementos.

Erro 3 — confundir operação entre arrays com concatenação

Em listas Python, `+` costuma ser entendido como junção de listas em alguns contextos:

```
lista_a = [1, 2, 3]
lista_b = [4, 5, 6]

print(lista_a + lista_b)
```

Saída:

```
[1, 2, 3, 4, 5, 6]
```

Em NumPy, a operação é matemática:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
```

Saída:

```
[5 7 9]
```

Isso precisa ficar muito claro.

23. Consolidação final da aula

NumPy representa um salto importante na forma de trabalhar com números em Python.

Até esse ponto, o uso de listas, laços e operações simples permite resolver muitos problemas básicos. No entanto, quando a necessidade passa a ser manipular conjuntos maiores de dados numéricos, aplicar cálculos em massa e organizar estruturas de forma matemática, o uso de uma biblioteca especializada deixa de ser opcional e passa a ser necessário.

Ao longo desta aula foram consolidados os seguintes pilares:

- arrays como estruturas numéricas otimizadas;
- operações vetorizadas sem necessidade de for;
- interação matemática entre arrays;
- geração automática de sequências;
- criação de estruturas especiais;
- uso de funções estatísticas prontas;
- reorganização estrutural com reshape;
- acesso a valores, faixas, linhas e colunas;
- filtragem de dados por condição;
- aplicação prática em contextos de análise.

NumPy não deve ser visto apenas como uma biblioteca para “contas com arrays”. Ele é a base de uma forma mais profissional, mais científica e mais escalável de tratar números em Python.

Na continuação natural desse desenvolvimento, a biblioteca deixa de ser usada apenas para criar arrays e passa a servir como ferramenta de manipulação mais profunda de estruturas numéricas, transformações e processamento de dados com maior complexidade.